



PROYECTO GESTIÓN DE GIT Y GITHUB

ISO P411



16 DE ABRIL DE 2026

ANDRÉS VIZCAÍNO BAZAGA

DIRIGIDO AL PROFESOR DON ALFREDO ABAD DOMINGO

ÍNDICE

Introducción	2
Paso 1: Preparación del entorno en nuestra VM	3
Paso 2: Configuración de nuestra identidad en Git	5
Paso 3: Preparación de la cuenta en GitHub y creación del repositorio remoto	6
Paso 4: Inicialización del repositorio local	7
Paso 5: Creación de nuestro primer "Commit"	8
Paso 6: Conectar Git local con GitHub (El "Push")	10
Paso 7: Trabajando con Ramas (Branches) y preparando el Pull Request	11
Paso 8: Creación y Fusión del Pull Request en GitHub	12
Conclusión	14

Introducción

Nuestra misión no es solo aprender la teoría, sino construir un tutorial práctico basado en nuestra propia experiencia de ejecución.

Nuestros objetivos principales son:

- **Estudiar Git** como sistema de control de versiones local.
- **Estudiar GitHub** como plataforma de alojamiento y colaboración.
- **Configurar y gestionar la relación** técnica entre ambas herramientas para sincronizar nuestro trabajo.

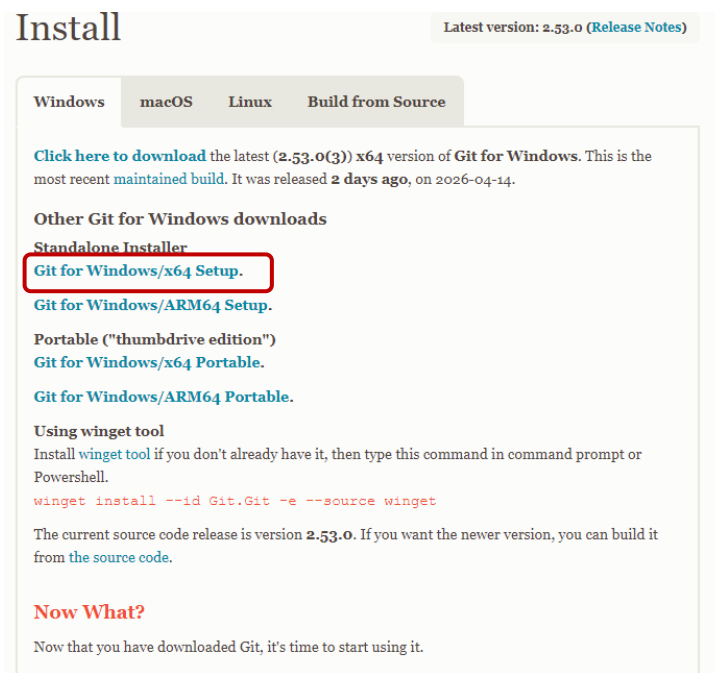
Convertiremos la documentación de este curso en un repositorio funcional, integrando guías externas para fortalecer nuestro conocimiento sobre flujos de trabajo como los *Pull Requests*. Al finalizar, tendremos un repositorio público y un documento Word que avale cada uno de nuestros pasos.

Paso 1: Preparación del entorno en nuestra VM

Ahora que tenemos nuestra máquina virtual de Windows 10 encendida, lo primero que necesitamos es "el motor": **Git**.

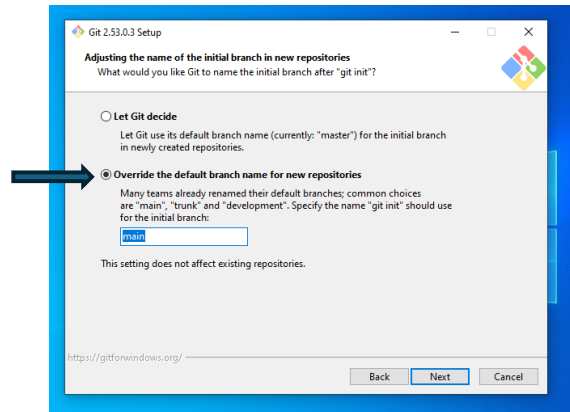
Lo que vamos a hacer:

1. Abriremos el navegador dentro de nuestra máquina virtual.
2. Iremos a la página oficial de [Git para Windows](#).
3. Descargaremos el instalador de 64 bits.

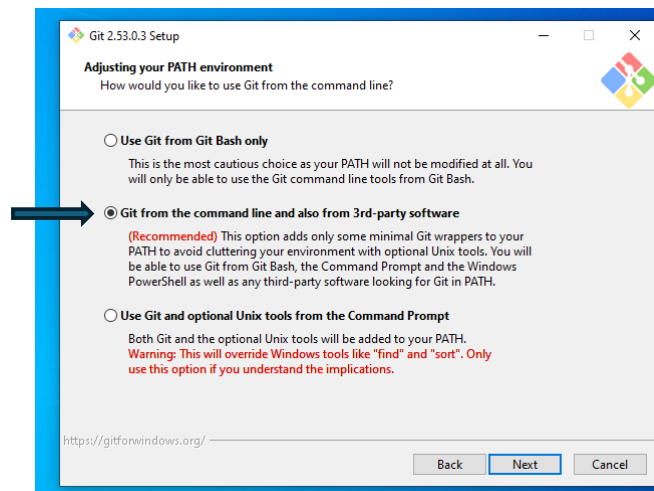


4. Ejecución: Iniciaremos el instalador. Durante el proceso, nos aseguraremos de:

- Dejar las opciones por defecto en la mayoría de los pasos. Excepto en los siguientes:
 - Cuando nos pregunte por el nombre de la rama inicial para nuevos repositorios, seleccionaremos la opción que nos permite elegir el nombre (recomendamos escribir **"main"**).



- Mantener la opción de usar Git desde la línea de comandos y también desde software de terceros.

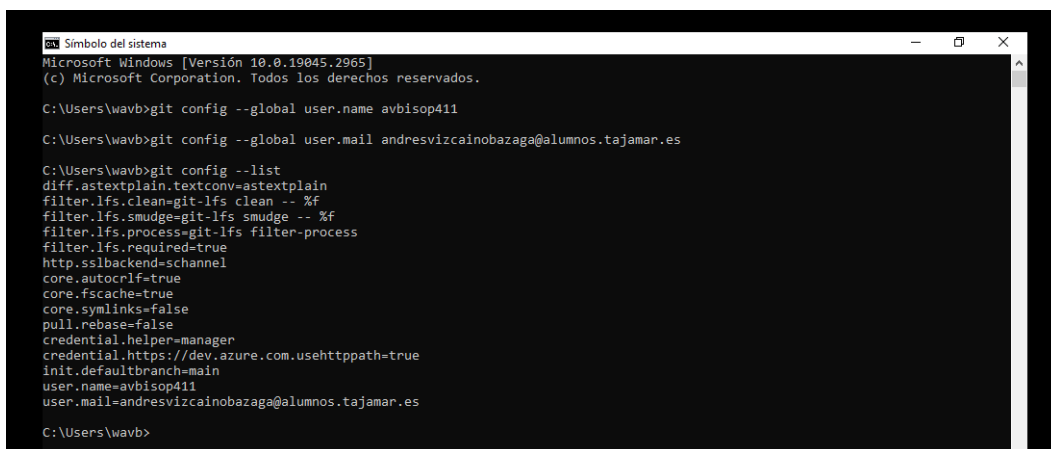


Paso 2: Configuración de nuestra identidad en Git

En este paso vamos a configurar nuestro nombre y correo electrónico de manera global. Esto es vital porque Git asocia esta información a cada "commit" (guardado de cambios) que realicemos.

Lo que vamos a hacer nosotros:

1. **Abrir la terminal:** Usaremos la que más nos guste (CMD o Git Bash).
2. **Configurar el nombre:** Escribiremos el siguiente comando:
 - `git config --global user.name "Nuestro Nombre"`
3. **Configurar el correo:** Usaremos el mismo correo con el que nos registraremos (o ya estamos registrados) en GitHub:
 - `git config --global user.email "nuestrocorreo@ejemplo.com"`
4. **Verificar la configuración:** Para estar seguros de que todo se guardó bien, lanzaremos:
 - `git config --list`



```
Símbolo del sistema
Microsoft Windows [Versión 10.0.19045.2965]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\wavyb>git config --global user.name avbisop411

C:\Users\wavyb>git config --global user.mail andresvizcainobazaga@alumnos.tajamar.es

C:\Users\wavyb>git config --list
diff.astextplain.textconv=astextplain
filter.lfs.clean=git-lfs clean -- %f
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
http.sslbackend=schannel
core.autocrlf=true
core.fscache=true
core.symlinks=false
pull.rebase=false
credential.helper=manager
credential.https://dev.azure.com.usehttppath=true
init.defaultbranch=main
user.name=avbisop411
user.mail=andresvizcainobazaga@alumnos.tajamar.es

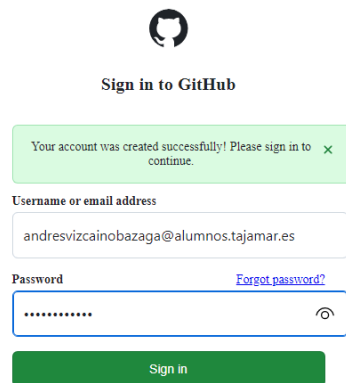
C:\Users\wavyb>
```

Paso 3: Preparación de la cuenta en GitHub y creación del repositorio remoto

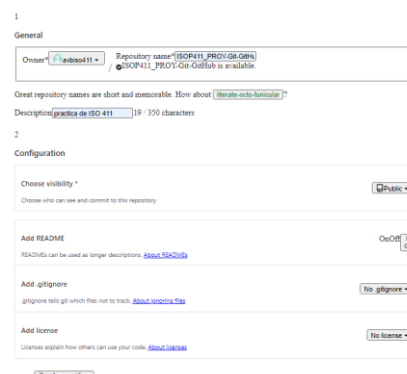
Para que nuestro tutorial sea completo, necesitamos el lugar donde alojaremos nuestro trabajo en la nube. Según las instrucciones, debemos usar una **cuenta gratuita y pública de GitHub**.

Los pasos a seguir son los siguientes:

1. **Acceso a GitHub:** Entraremos en github.com desde el navegador de nuestra máquina virtual.
2. **Registro/Log-in:** Si no tenemos cuenta, la crearemos. Si ya la tenemos, iniciaremos sesión.



3. **Crear Nuevo Repositorio:** * Haremos clic en el botón **"New"** (Nuevo).
 - Le pondremos el nombre solicitado para nuestra práctica: ISOP411_PROY-Git-GitHub.
 - Nos aseguraremos de que esté marcado como **Público**.
 - **Importante:** Por ahora, no lo inicializaremos con README ni .gitignore, ya que queremos aprender a vincularlo desde nuestra máquina local manualmente.



Paso 4: Inicialización del repositorio local

Nuestras acciones en la terminal:

1. **Crear nuestra carpeta de trabajo:**

- `mkdir Practica-Git`

2. **Entrar en la carpeta:**

- `cd Practica-Git`

3. **Inicializar Git:** Este es el comando clave que crea el "cerebro" de Git (la carpeta oculta `.git`) en nuestro directorio local.

- `git init`

```
C:\Users\wavyb>mkdir Practica-Git
C:\Users\wavyb>cd Practica-Git
C:\Users\wavyb\Practica-Git>git init
Initialized empty Git repository in C:/Users/wavyb/Practica-Git/.git/
C:\Users\wavyb\Practica-Git>
```

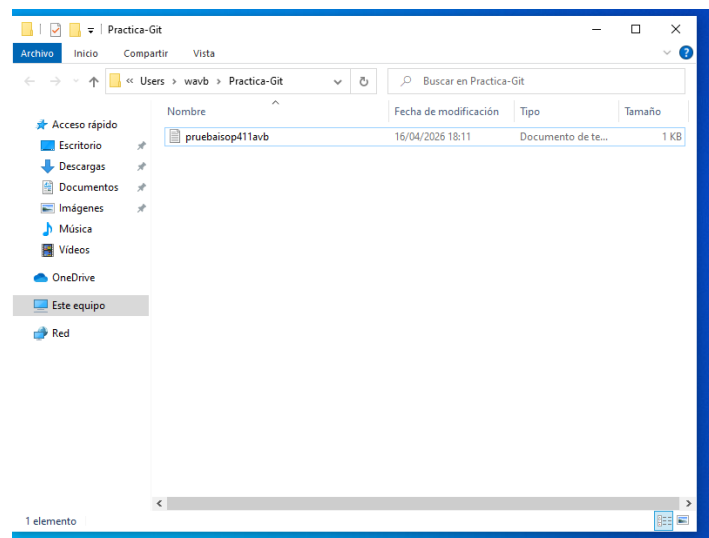

Paso 5: Creación de nuestro primer "Commit"

En Git, un "commit" es como una fotografía o un punto de guardado de nuestros archivos. Para hacerlo, seguimos el flujo de trabajo básico de Git: modificar, añadir (add) y guardar (commit).

Lo que vamos a hacer nosotros en nuestra máquina virtual:

1. **Crear el archivo de documentación:** Dentro de nuestra carpeta Practica-Git, crearemos un archivo de texto normal.

Lo llamaremos *"pruebaisop411"* y escribiremos dentro algo sencillo, como: *"Hola esto es una prueba de AVB"*.



2. **Comprobar el estado:** Volvemos a la terminal y escribimos:

- `git status`
(Veremos nuestro archivo en rojo, lo que significa que Git lo ve, pero aún no lo está rastreando).

```
C:\Users\wavb\Practica-Git>git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
  pruebaisop411avb.txt

nothing added to commit but untracked files present (use "git add" to track)
```

3. **Añadir al área de preparación (Staging):** Para decirle a Git que queremos incluir este archivo en nuestro próximo guardado, usamos:

- `git add pruebaisop411.txt`

4. **Crear el commit:** Ahora guardamos el cambio en el historial de Git con un mensaje descriptivo:

- `git commit -m "Primer commit: añadida documentacion del curso"`

```
C:\Users\wavyb\Practica-Git>git commit -m "Primer commit: añadida documentacion del curso"
[main (root-commit) 6cf1e57] Primer commit: añadida documentacion del curso
1 file changed, 1 insertion(+)
create mode 100644 pruebaisop411avb.txt
```

Paso 6: Conectar Git local con GitHub (El "Push")

Este es uno de los momentos más importantes de la práctica, ya que vamos a cumplir con el objetivo de aprender a configurar y gestionar la relación de Git con GitHub.

Vamos a coger lo que tenemos en nuestro PC y lo vamos a enviar al repositorio remoto que creamos en el Paso 3 con la nomenclatura ISOP411_PROY-Git-GitHub.

Los pasos a seguir son los siguientes:

1. **Copiar la URL:** Iremos a la pestaña de nuestro navegador donde tenemos abierto nuestro repositorio de GitHub. Allí veremos un botón verde que dice "Code" y copiaremos la URL web.
2. **Vincular el origen remoto:** Volveremos a nuestra terminal y le diremos a Git dónde está la nube ejecutando:
 - `git remote add origin pegamos_nuestra_url_aqui`
3. **Subir los cambios (Push):** Finalmente, enviaremos nuestro archivo `pruebasop411avb.txt` a GitHub con el siguiente comando:
 - `git push -u origin main`

```
C:\Users\wavb\Practica-Git>git push -u origin main
info: please complete authentication in your browser...
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 290 bytes | 72.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/avbiso411/ISOP411_PROY-Git-GitHub.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.

C:\Users\wavb\Practica-Git>git chekout -b mejora-documentacion
git: 'chekout' is not a git command. See 'git --help'.

The most similar command is
  checkout

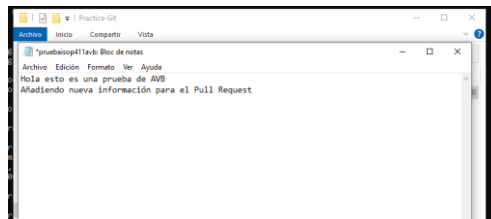
C:\Users\wavb\Practica-Git>
```

Paso 7: Trabajando con Ramas (Branches) y preparando el Pull Request

Un Pull Request consiste en proponer cambios desde una versión paralela de nuestro proyecto (una "rama") hacia la versión principal (la rama "main"). Vamos a simular que estamos añadiendo una mejora a nuestra documentación.

Lo que vamos a hacer nosotros en la terminal:

1. **Crear y movernos a una nueva rama:** Vamos a crear una rama paralela llamada mejora-documentacion y nos cambiaremos a ella en un solo paso:
 - `git checkout -b mejora-documentacion`
2. **Modificar nuestro archivo:** Vamos a abrir el archivo que creamos antes (pruebaisop411avb.txt) con el bloc de notas y le vamos a añadir una segunda línea de texto que diga, por ejemplo: *"Añadiendo nueva información para el Pull Request"*. Lo guardamos y cerramos.



3. **Añadir y guardar (Commit):** Como ya sabemos hacerlo, vamos a registrar este cambio en nuestra nueva rama:
 - `git add pruebaisop411avb.txt`
 - `git commit -m "Mejora: añadida segunda linea a la documentacion"`
4. **Subir nuestra nueva rama a GitHub (Push):** Ahora enviamos esta rama paralela a la nube:
 - `git push -u origin mejora-documentacion`

```
C:\Users\wavy\Practica-Git>git checkout -b mejora-documentacion
git: 'checkout' is not a git command. See 'git --help'.

The most similar command is
  checkout

C:\Users\wavy\Practica-Git>git checkout -b mejora-documentacion
Switched to a new branch 'mejora-documentacion'

C:\Users\wavy\Practica-Git>git add pruebaisop411avb.txt

C:\Users\wavy\Practica-Git>git commit -m "Mejora: añadida segunda linea a la documentacion"
[mejora-documentacion 72b9d7e] Mejora: añadida segunda linea a la documentacion
1 file changed, 2 insertions(+), 1 deletion(-)

C:\Users\wavy\Practica-Git>git push -u origin mejora-documentacion
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 375 bytes | 375.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'mejora-documentacion' on GitHub by visiting:
remote:   https://github.com/avbiso411/ISOP411_PROY-Git-GitHub/pull/new/mejora-documentacion
remote:
To https://github.com/avbiso411/ISOP411_PROY-Git-GitHub.git
 * [new branch]   mejora-documentacion -> mejora-documentacion
branch 'mejora-documentacion' set up to track 'origin/mejora-documentacion'.

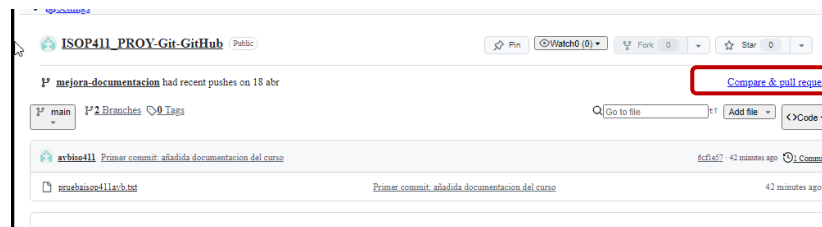
C:\Users\wavy\Practica-Git>
```

Paso 8: Creación y Fusión del Pull Request en GitHub

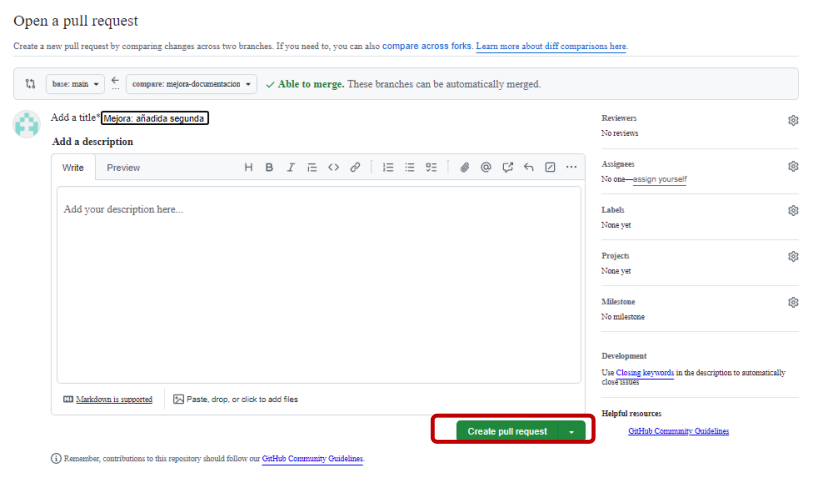
Todo este paso lo vamos a realizar desde el navegador, en la página web de GitHub, ya que es la plataforma la que nos facilita esta revisión de código.

Pasos a seguir:

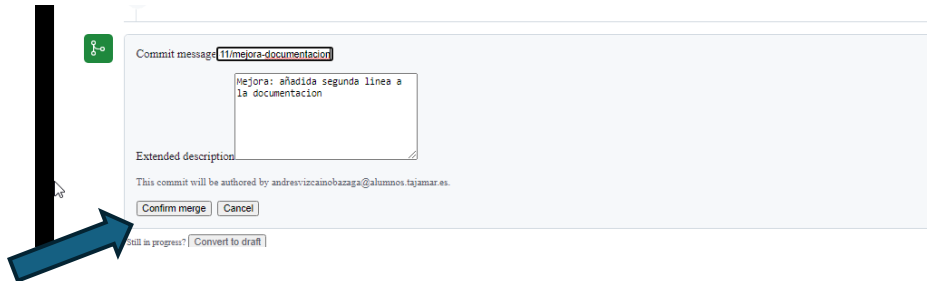
1. **Ir a GitHub:** Abrimos nuestro navegador en la máquina virtual y entramos en la página de nuestro repositorio (ISOP411_PROY-Git-GitHub).
2. **Detectar el aviso:** En la parte superior con un botón que dice **"Compare & pull request"**. Haremos clic ahí.



3. **Crear la petición:** Nos aparecerá una pantalla donde podemos poner un título y un comentario explicando qué hemos cambiado. Dejamos el título que viene por defecto y le damos al botón verde **"Create pull request"**.



4. **Fusionar (Merge):** Como somos los dueños del repositorio y sabemos que el código está bien, ahora vamos a aceptarlo. Hacemos clic en el botón verde que dice **"Merge pull request"** y luego en **"Confirm merge"**.



Al hacer esto, el icono cambiará a un color morado que dice "Merged", lo que significa que el texto de nuestra rama paralela se ha unido con éxito a la rama main.



Conclusión

A lo largo del desarrollo de esta práctica, hemos logrado cumplir con éxito todos los objetivos propuestos para la gestión y control de versiones.

Comenzamos preparando nuestro entorno de trabajo local mediante la instalación y configuración de **Git**, lo que nos permitió comprender la importancia de registrar la autoría y el historial de cambios en nuestros archivos a través de los **commits**.

Posteriormente, dimos el salto a la nube creando nuestro repositorio público en **GitHub**.

Finalmente, para afianzar nuestras habilidades en entornos colaborativos, fuimos un paso más allá del flujo básico: creamos ramas de desarrollo paralelas, realizamos modificaciones aisladas y llevamos a cabo un **Pull Request**, culminando con la fusión (*Merge*) exitosa en nuestra rama principal.

Esta práctica nos ha servido entre otras cosas para proporcionarnos una base técnica sólida y fundamental que aplicaremos en cualquier proyecto de desarrollo de software profesional en el futuro.

https://github.com/avbiso411/ISOP411_PROY-Git-GitHub/pull/1